Lecture 1

Web Application Basics and Flask Introduction

- Course: Web Application Vulnerability Lab
- Theme: Understanding the basic structure of web applications
- Goal: Be able to start a Flask app and explain the relationship between pages and code

Learning Goals for Today

- Explain that a web application works through a browser and a server
- Explain the flow of HTTP requests and responses
- Start a Flask application locally
- Read the relationship between pages and routing in the code
- Understand the overall structure of the lab application used in later classes

Topics for Today

1. Web application basics
2. HTTP basics
3. What Flask is
4. Getting the lab material from GitHub
5. Exploring the application in the browser
6. How to read the code
7. Exercises

What Is a Web Application?

- The browser sends a request to the server
- The server processes the request and returns a result
- The browser displays the returned HTML

Example:

- Open `/login`
- Submit the login form
- The server authenticates the user and returns `/me`

Important Terms

- Client:
  - The browser
- Server:
  - The Flask application
- Request:
  - A message sent from the browser
- Response:
  - A message returned by the server
- Routing:
  - The mechanism that selects code based on the URL

Minimal HTTP Image

The browser opens a URL:

```
GET /login HTTP/1.1
```

The server returns a page:

```
HTTP/1.1 200 OK
Content-Type: text/html
```

Form submission:

```
POST /login
```

GET and POST

- `GET`
  - Used mainly for page display and data retrieval
  - Common when opening a URL
- `POST`
  - Used mainly for actions that change state
  - Login, logout, post creation, and similar actions

In today's app:

- `GET /login`
- `POST /login`
- `POST /logout`

What Is Flask?

- A lightweight web framework for Python
- Lets us build web applications with a small amount of code
- Makes routing and templates relatively easy to understand
- Good for teaching because it is easier to see what is happening

In this course:

- We use Flask to read and run the lab application
- Then we build vulnerability exercises on top of it

The Lab Application Used in This Course

This application will be used step by step throughout the course.

- Login
- Logout
- Authenticated pages
- User search
- Message board
- Profile update
- lab-settings

Today's focus:

- Being able to start the app
- Understanding how pages match the code

Where to Get the Material

The lab material is provided through a public GitHub repository.

- Repository:
  - https://github.com/koide55/LectureWebApp001

In class, we will clone this repository to the local machine.

First Step

```
git clone https://github.com/koide55/LectureWebApp001.git
cd LectureWebApp001
```

After that, create the Python environment and start the application.

Directory Structure

```
LectureWebApp001/
  app/
    __init__.py
    routes.py
    templates/
    services/
    db/
  run.py
  requirements.txt
```

Important files:

- `run.py` : application entry point
- `app/__init__.py` : Flask application factory
- `app/routes.py` : page handlers
- `app/templates/` : HTML templates

How to Start the Application

```
git clone https://github.com/koide55/LectureWebApp001.git
cd LectureWebApp001
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python3 -m app.db.seed
python3 run.py
```

Open this in the browser:

- http://127.0.0.1:5000

Default Users

- `alice / alicepass`
- `bob / bobpass`
- `admin / adminpass`

Start by logging in as `alice`.

First Hands-On

1. Clone the repository
2. Create a virtual environment
3. Install the dependencies
4. Run the seed command
5. Start the application
6. Open the top page
7. Log in with `alice / alicepass`
8. Open `My Page` and `Lab Settings`

What to observe:

- What each command is doing
- How the URL changes
- How each page has a different role
- How the behavior changes before and after login

Code Entry Point 1

`run.py`

```python
from app import create_app

app = create_app()

if __name__ == "__main__":
    app.run(debug=True)
```

Key points:

- `create_app()` creates the Flask application
- `app.run()` starts the development server

Code Entry Point 2

`app/__init__.py`

```python
def create_app(config_class=None):
    from flask import Flask
    from .config import Config
    from .routes import main_bp

    if config_class is None:
        config_class = Config

    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object(config_class)
    app.register_blueprint(main_bp)
    return app
```

Key points:

- Creates the Flask application instance
- Loads configuration

Code Entry Point 3

`app/routes.py`

```
main_bp = Blueprint("main", __name__)

@main_bp.get("/")
def index():
    return render_template("index.html")
```

Key points:

- Accessing `"/"` runs `index()`
- It returns `index.html`
- The URL and the function correspond to each other

How to Read Routing

Example:

```
@main_bp.route("/login", methods=["GET", "POST"])
def login():
    ...
```

Meaning:

- It handles access to `/login`
- `GET` is used to display the page
- `POST` is used to process form submission

The Role of Templates

`render_template("login.html")` returns HTML generated from `app/templates/login.html`.

So:

- Python:
  - Handles the program flow
- HTML templates:
  - Handle the page layout and display

Code Explanation

`login.html`

```
<form action="{{ url_for('main.login') }}" method="post" class="stack">
  <label for="username">Username</label>
  <input id="username" name="username" type="text" required>

  <label for="password">Password</label>
  <input id="password" name="password" type="password" required>

  <button type="submit">Login</button>
</form>
```

Key points:

- `form` defines a submission form
- `method="post"` sends the form using POST
- Names such as `username` are the keys used on the server side

Form Submission and Server-Side Processing

```
user, error, cookie_value = attempt_login(
    request.form.get("username", "").strip(),
    request.form.get("password", ""),
)
```

Key points:

- The server reads values from `request.form`
- Authentication happens on the server side
- Typing something into the form does not mean you are logged in yet

Matching Pages and Code

By the end of today, you should be able to explain the following matches.

- Top page:
  - `index.html`
  - `index()`
- Login page:
  - `login.html`
  - `login()`
- My page:
  - `me.html`
  - `me()`

What We Will Learn Later with This App

- Authentication
- Sessions
- Cookies
- SQL injection
- CSRF
- Reflected XSS
- Stored XSS
- Command injection
- Authorization issues

At this stage:

- It is more important to understand it first as a normal web application

Exercise 1

Clone the Repository and Start the App

Run the following commands.

```
git clone https://github.com/koide55/LectureWebApp001.git
cd LectureWebApp001
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python3 -m app.db.seed
python3 run.py
```

Then open the top page in your browser.

Exercise 2

Check the Relationship Between Pages and URLs

Fill in the table below.

Page	URL	Main role
Top page		
Login		
My Page		
Profile		
Board		
Lab Settings		

Exercise 3

Find the Routing

Open `app/routes.py` and find the code for:

1. `/`
2. `/login`
3. `/me`
4. `/profile`
5. `/lab-settings`

Check:

- Which function handles it
- Whether it uses `GET` or `POST`
- Which template it returns

Exercise 4

Read the Templates

Open the following templates and compare them.

- `app/templates/base.html`
- `app/templates/login.html`
- `app/templates/me.html`

Look at:

- Navigation
- Forms
- The displayed data

Exercise 5

Follow the Code

Answer the following questions.

1. Where is the application created?
2. Where is the display logic for `/login`?
3. Where is the submission logic for `/login`?
4. Which template is used by `/me`?

Exercise 6

Check the Behavior

Try the following in the browser.

1. Open `/me` before logging in
2. Open `/me` after logging in
3. Open `Profile`
4. Open `Lab Settings`

Think about:

- What changes before and after login
- Why that change happens

Summary of Today's Code Reading

- `run.py`
  - Entry point
- `app/__init__.py`
  - Application factory
- `app/routes.py`
  - Mapping between URLs and processing
- `app/templates/*.html`
  - Page structure and display

In short:

- A web app can be read through the relationship between URL, processing, and page

Next Time

- How login and logout work
- The difference between authentication and authorization
- How protected pages are enforced

Homework

1. Read `app/routes.py` and write down at least 5 URL-to-function mappings
2. Read `app/templates/base.html` and list the elements shared by all pages
3. Think about why `/login` uses both `GET` and `POST`

Instructor Notes

- In the first lecture, do not go too deep into vulnerabilities yet
- Focus on understanding the app first as a normal web application
- If there is time, show form submission in browser developer tools
- Carefully connect `/login` and `/me` so the next lecture on authentication is easier